

Go Toolchain & Program Basics Cheatsheet

Lessons: [01 — Introduction](#) · [02 — Environment Setup](#) · [03 — First Program](#)

Examples: — (this track starts at [04](#))

Covers: the `go` command, modules, environment variables, the program skeleton, declarations, doc comments

Legend: `[*]` = real Go API/flag that the lessons have not covered yet

VERSION & INSTALL

```
go version                which toolchain is on PATH
go version -m ./bin/app  [*] module + build info baked into a binary
go env                    every setting the toolchain resolved
go env GOPATH GOROOT     print just those
go env -w GOFLAGS=-mod=mod [*] persist a setting (writes to go/env)
go env -u GOFLAGS        [*] undo a -w setting
brew install go           macOS install; or download from go.dev/dl
go telemetry off         [*] Go 1.23+: opt out of local telemetry
```

THE go COMMAND: everyday

```
go run .                  compile + run the package in this dir
go run ./cmd/api          run a specific main package
go build                  binary named after the module/dir, in cwd
go build -o bin/app ./cmd/api choose the output path
go test ./...             test every package below here
go test -run TestName ./pkg only tests matching the regexp
go vet ./...              static checks the compiler doesn't do
go fmt ./...              run gofmt -l -w on the packages
gofmt -d file.go          [*] show the diff instead of rewriting
go doc strings.Builder    docs for a symbol, in the terminal
go doc -all strings       [*] the whole package's docs
go clean -cache           [*] wipe the build cache
```

THE go COMMAND: modules & deps

```
go mod init example.com/proj create go.mod with that module path
go mod tidy                  add what's imported, drop what isn't
go mod download              [*] fill the module cache without building
go mod verify                [*] check the cache against go.sum
go mod why <pkg>             [*] explain why a dependency is needed
go mod graph                 [*] print the module requirement graph
go mod edit -require=x@v1.2.3 [*] edit go.mod mechanically
go mod vendor                [*] copy deps into ./vendor
go get example.com/lib@v1.4.0 add/upgrade a dependency
go get -u ./...              [*] upgrade deps to latest minor/patch
go get example.com/lib@none  [*] drop a dependency
go install example.com/cmd@latest build & put a tool in $GOBIN
go list -m all               [*] every module in the build list
go work init ./a ./b        [*] workspaces: several modules, one build
```

THE go COMMAND: quality & release

go test -race ./...	data-race detector
go test -cover ./...	statement coverage per package
go test -bench=. ./...	run benchmarks
go build -ldflags "-s -w" [*]	strip symbols/DWARF -> smaller binary
GOOS=linux GOARCH=amd64 go build [*]	cross-compile
CGO_ENABLED=0 go build [*]	fully static binary
go tool dist list	[*] every GOOS/GOARCH pair
go tool pprof <profile>	[*] open a CPU/heap profile
golangci-lint run	[*] the aggregate linter (not stdlib)
govulncheck ./...	[*] known vulnerabilities in your deps

ENVIRONMENT VARIABLES

GOROOT	where the toolchain itself lives (leave alone)
GOPATH	~/go by default: bin/, pkg/mod/, src/
GOMODCACHE	\$GOPATH/pkg/mod – the downloaded modules
GOBIN	where `go install` puts binaries
GOOS / GOARCH	[*] target OS/CPU for the build
GOFLAGS	[*] flags implicitly added to every go command
GOPROXY	[*] module proxy (proxy.golang.org; `direct`; `off`)
GOPRIVATE	[*] module prefixes that skip proxy + checksum db
GONOSUMDB / GONOSUMCHECK	[*] older names for checksum exemptions
GODEBUG	[*] runtime debug knobs (e.g. netdns=go)

go.mod & go.sum

module example.com/proj	the import path this module answers to
go 1.24	language version the module is written for
toolchain go1.26.0	[*] pin the toolchain that builds it
require (...)	direct + indirect dependencies
// indirect	a dep of a dep, not imported by you
replace x => ./local	[*] point a module at a fork or local dir
exclude x v1.2.3	[*] refuse one specific version
go.sum	cryptographic hashes of every module used
(commit both files; go.sum is not a lock file – the build list is)	

PROGRAM SKELETON

package main	an executable's package name
func main()	the entry point: no args, no return
import "fmt"	one import
import ("fmt"; "os")	grouped: stdlib first, blank line, then external
import f "fmt"	[*] alias an import
import _ "net/http/pprof"	[*] blank import: run init() only, no symbols
func init()	[*] runs before main, after package vars
os.Exit(1)	exit with a status (defers do NOT run)

DECLARATIONS

<code>var x int</code>	declared, zero value (0)
<code>var x int = 5</code>	explicit type + value
<code>var x = 5</code>	type inferred → int
<code>x := 5</code>	short form; only inside a function
<code>var (a = 1; b = 2)</code>	grouped declaration
<code>const Pi = 3.14159</code>	untyped constant, arbitrary precision
<code>const MaxUsers int = 100</code>	typed constant
<code>const big = 1 << 30</code>	constant expression, evaluated at compile time
<code>a, b := 1, 2</code>	multiple assignment
<code>a, b = b, a</code>	swap, no temp needed
<code>_ = someValue</code>	discard with the blank identifier

COMMENTS & DOC

<code>// line comment</code>	the normal one
<code>/* block comment */</code>	rarely used; no nesting
<code>// Foo does X.</code>	doc comment: starts with the symbol's name
<code>// Deprecated: use Bar.</code>	[*] recognized by tooling
<code>//go:embed file.txt</code>	[*] directive, no space after //
<code>//go:generate cmd args</code>	[*] run by `go generate`
<code>Package foo ...</code>	package doc lives above `package foo`

TRAPS & MEMORIZE

<code>:=</code> outside a function	compile error – use var at package level
unused import / variable	compile error, not a warning
gofmt uses tabs	never argue with it; there are no options
capital = exported	Name is public, name is package-private
one main() per main package	and `package main` is required for a binary
go run does not install	the binary lands in a temp dir and is deleted
module path ≠ folder name	imports follow the module path in go.mod
go get no longer builds	use `go install pkg@version` for tools
os.Exit skips defers	and skips buffered output flushes
GOPATH mode is dead	modules everywhere since Go 1.16